claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf_133acdee-dab\agent-a48b892b8e66fe313.jsonl`
- SHA-256: `6effa9f5400b3f5bcd9489a706bff806d8bcf14b4bcc456aab4636858e9aaf2b`
- Source modified: `2026-07-06T17:29:38+00:00`
- Imported at: `2026-07-08T15:59:35+00:00`
- project: `wf_133acdee-dab`
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`

Transcript

1. user (2026-07-06T17:29:03.348Z)

You are an ADVERSARIAL VERIFIER. Try HARD to REFUTE the following review finding by reading the ACTUAL code in C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (and the diff at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_fix.diff). Default to REFUTED if you cannot
concretely reproduce the defect by pointing at real code lines. A finding is CONFIRMED only if you can trace a concrete input/state that produces the claimed wrong behavior in the code AS WRITTEN (not as you imagine it). If the claimed line/behavior does not exist, or a guard/caller already handles it, or it is out-of-scope speculation, mark REFUTED. If genuinely unclear after reading, mark UNCERTAIN.

FINDING (from the completeness lens):
severity=high | backend/pipeline/segmenters/trajectory_hybrid.py:219
summary: The missing-API-key abort still returns a bare [] instead of a Failure, so an unrunnable config surfaces as a 0-segment status="success" ? the exact wf294 silent-failure class the fix set out to close.
detail: process_video has three pre-detection config-abort paths that STILL return bare []: unknown sport (line 194, KeyError), missing ${PROVIDER}_API_KEY (line 219), and unknown provider (line 226, KeyError). The missing-key case is the most acute: the worker's OWN 0-segment diagnostic warning in backend/worker/__main__.py:340-352 explicitly names it as a top cause of a false 0-segment 'success' ('a missing ${AI_PROVIDER}_API_KEY with skip_ai_handoff=False (the segmenter bails BEFORE detection)'). The sibling gemini segmenter already treats exactly these three conditions as a Failure ? backend/pipeline/segmenters/gemini.py:100/116/124, whose docstring cites 'Finding B ? the loud-failures bar ... not a silent 0-rally success that looks like an empty match'. motion.py:39 likewise returns Failure on 'could not open video'. The wf294 fix widened the return type to Union[List, Failure] and converted the two DETECTOR paths, but left these config-abort paths as bare [], so they remain inconsistent with the codebase's own established Failure convention and still leak as status="success" through _serialize_and_push_outputs. A cloud run submitted before the WASB env has a key set reproduces the wf294 shape verbatim.
failure_scenario: A wasb_hybrid/tracknet_hybrid/fusion_hybrid cloud job runs with skip_ai_handoff=False but GEMINI_API_KEY unset (or an unknown sport/provider in config). process_video returns bare [] at line 219 (or 194/226). cmd_infer's isinstance(result, Failure) is False, so status is set to 'success', _serialize_and_push_outputs writes report.json status="success"/segment_count=0, and the done-marker carries rc=0/status="success". orchestration.cached_process_result (line 326) and probe_process_cache (line 280) accept the report as a usable success, and _resume_one_remote_job (job_worker.py:462-472) marks the job DONE with 0 rallies ? a misconfig indistinguishable from a legitimately empty match, the exact incident.
suggested_fix: Return a Failure (is_recoverable=True) from the missing-API-key (line 219), unknown-sport (line 194), and unknown-provider (line 226) paths, matching gemini._resolve_provider. Consider also the video_duration<=0 (line 231) and _resolve_runner NotImplementedError (line 244) paths, which are input/env failures rather than empty matches. Only the trajectory-produced-but-zero-windows return (line 371) is a genuine legit-empty [].

Verify against the real code and return your verdict.

2. assistant (2026-07-06T17:29:05.331Z)

I'll verify this finding by reading the actual code.

3. user (2026-07-06T17:29:07.344Z)

```
170             Persisted candidates are NOT affected ? they remain the full superset for
offline
171             re-scoring."""
172             return list(range(len(windows)))
173
174         def process_video(  # type: ignore[override]
175             self,
176             video_id: str,
177             video_path: str,
178             player_pool: Optional[List[str]] = None,
179             max_frames: Optional[int] = None,
180         ) -> "Union[List[Dict[str, Any]], Failure]":
181             # Partial adoption of the Result contract (the ABC declares Success|Failure).
182             # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
/
183             # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused with
a
184             # genuine 0-rally video (which still returns a bare ``[]``). Every caller
185             # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
Failure)``;
186             # the bare-list success/empty paths are the documented incremental gap (CODE_MAP
gotchas).
187             label = self.display_label
188             # --- Sport profile + AI provider wiring (identical across segmenters) ---
189             sport_name = self.config.sport
190             try:
191                 sport_profile = sport_registry.get(sport_name)
192             except KeyError as e:
193                 logger.error(str(e))
194                 return []
195
196             idx_cfg = self.config.indexing
197             skip_ai = bool(idx_cfg.skip_ai_handoff)
198             provider_name = self.config.ai_provider
199             if skip_ai:
200                 model_name = "local-noai"
201                 logger.info(
202                     "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
203                     "be emitted as rallies; no %s handoff, no API key needed.",
204                     label,
205                     provider_name,
206                 )
207             else:
208                 model_name = self.config.ai_model
209                 api_key_env_var = f"{provider_name.upper()}_API_KEY"
210                 api_key = os.environ.get(api_key_env_var)
211                 if not api_key:
212                     from backend.pipeline.segmenters._keyhint import api_key_hint
213
214                     logger.error(
215                         f"{api_key_env_var} environment variable is not set! "
216                         f"To run the {provider_name} handoff, set your API key. "
217                         + api_key_hint(api_key_env_var)
218                     )
219                     return []
220                 try:
221                     provider = provider_registry.get(
```

```
                        provider_name, api_key=api_key, model_name=model_name
                    )
                except KeyError as e:
                    logger.error(str(e))
                    return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

            # --- Runner config + windowing thresholds ---
            # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
            # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
            # it delegates to the subclass _build_runner (the real WSL/native runner).
            try:
                runner, runner_params = self._resolve_runner(idx_cfg)
            except NotImplementedError as e:
                # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
                # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
                logger.error("%s: %s", label, e)
                return []

            velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
            merge_gap = idx_cfg.dead_time_merge_gap
            min_window_duration = idx_cfg.min_action_window_duration
            # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
            max_window_duration = idx_cfg.max_window_duration
            window_min_split_gap = idx_cfg.window_min_split_gap
            window_hard_cap = idx_cfg.window_hard_cap
            window_inactivity_end = idx_cfg.window_inactivity_end
            inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
            inactivity_min_density = idx_cfg.inactivity_min_density
            inactivity_temporal_density = idx_cfg.inactivity_temporal_density
            window_blob_fallback = idx_cfg.window_blob_fallback
            window_blob_factor = idx_cfg.window_blob_factor
            handoff_padding = idx_cfg.ai_handoff_padding
            ai_max_workers = idx_cfg.ai_max_workers
            log_candidates = idx_cfg.log_yolo_candidates
            store_candidates = idx_cfg.store_yolo_candidates

            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s detector environment not ready: %s", label, msg)
                # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
                # failed/retryable instead of writing an empty-but-"successful" result.
                return Failure(
                    message=f"{label} detector environment not ready: {msg}",
                    is_recoverable=True,
                )
            logger.info("%s detector env OK: %s", label, msg)
```

```
283
284             # --- Phase 2: trajectory -> candidate rally windows ---
285             # Trajectory detectors process the whole video in one pass (they carry
286             # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
287             t_start = time.time()
288             out_dir = os.path.join(output_base(self.config), self.family)
289             csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
290             if not csv_path:
291                 logger.error("%s produced no trajectory; aborting.", label)
292                 # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
293                 # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
294                 # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
295                 # rather than a silent 0-segment "success" (the wf294 incident).
296                 return Failure(
297                     message=(
298                         f"{label} detector produced no trajectory (timed out or aborted) ? "
299                         "detector failure, not a 0-rally video."
300                     ),
301                     is_recoverable=True,
302                 )
303
304             points = runner.parse_trajectory_csv(csv_path)
305             logger.info(
306                 "Parsed %d trajectory points (%d visible).",
307                 len(points),
308                 sum(1 for p in points if p.visible),
309             )
310
311             all_candidate_windows = runner.trajectory_to_action_windows(
312                 points,
313                 fps=fps,
314                 frame_width=frame_width,
315                 chunk_start=0.0,
316                 velocity_thresh=velocity_thresh,
317                 merge_gap=merge_gap,
318                 min_window_duration=min_window_duration,
319                 chunk_index=0,
320                 max_window_duration=max_window_duration,
321                 min_split_gap=window_min_split_gap,
322                 window_hard_cap=window_hard_cap,
323                 window_inactivity_end=window_inactivity_end,
324                 inactivity_rally_thresh=inactivity_rally_thresh,
325                 inactivity_min_density=inactivity_min_density,
326                 inactivity_temporal_density=inactivity_temporal_density,
327                 window_blob_fallback=window_blob_fallback,
328                 window_blob_factor=window_blob_factor,
329             )
330             logger.info(
331                 "Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
332                 label,
333                 time.time() - t_start,
334                 len(all_candidate_windows),
335             )
336
337             if log_candidates:
338                 for cw in all_candidate_windows:
339                     logger.info(
340                         f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
341                         f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
342                         f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
343                     )
```

```
344
345             # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
346             # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
347             # persistence and the handoff gate below.
348             window_stats = [
349                 self._enrich_candidate_stats(
350                     cw, points, fps, frame_width, video_path, idx_cfg
351                 )
352                 for cw in all_candidate_windows
353             ]
354
355             # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
356             candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
357             if store_candidates:
358                 for i, cw in enumerate(all_candidate_windows):
359                     candidate_ids[i] = self.db.add_candidate_segment(
360                         video_id,
361                         detector=self.name,
362                         start_time=cw["start"],
363                         end_time=cw["end"],
364                         chunk_index=cw["chunk_index"],
365                         confidence=min(1.0, cw["peak_velocity"]),
366                         stats=window_stats[i],
367                         detection_params=detection_params,
368                     )
369
370             if not all_candidate_windows:
371                 return []
372
373             # --- Phase 3: AI provider handoff over padded candidate clips ---
374             # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
375             # candidates above stay the full superset ? only the served play_segments are
gated.
376             handoff_indices = self._select_for_handoff(
377                 all_candidate_windows, window_stats, idx_cfg
378             )
379
380             ai_segments: List[dict] = []
381             if skip_ai:
382                 # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
383                 # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
384                 # falls back to the duration-based estimate (no AI exchange count).
385                 ai_segments = [
386                     {
387                         "start_time": all_candidate_windows[wi]["start"],
388                         "end_time": all_candidate_windows[wi]["end"],
389                         "shuttle_exchange_count": None,
```

4. user (2026-07-06T17:29:08.881Z)

```
1       diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
2       index 503f6d8..f96b495 100644
3       --- a/backend/pipeline/segmenters/trajectory_hybrid.py
4       +++ b/backend/pipeline/segmenters/trajectory_hybrid.py
5       @@ -22,7 +22,7 @@ import concurrent.futures
6        import logging
7        import os
8        import time
9       -from typing import Any, Dict, List, Optional, Tuple
```

```
+from typing import Any, Dict, List, Optional, Tuple, Union

 import cv2

@@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
 from backend.pipeline.detectors.base import DetectorRunner
 from backend.pipeline.segmenters.base import BasePlaySegmenter
 from backend.providers import provider_registry
+
+# Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
+from backend.results import Failure
 from backend.sports import sport_registry
 from backend.utils.paths import output_base
 from backend.utils.video import extract_video_chunk, get_video_duration
@@ -174,10 +177,13 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         video_path: str,
         player_pool: Optional[List[str]] = None,
         max_frames: Optional[int] = None,
-    ) -> List[Dict[str, Any]]:
-        # override-ignore: the ABC declares the Result contract (Success|Failure)
-        # but every live segmenter still returns a bare list ? the documented
-        # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
+    ) -> "Union[List[Dict[str, Any]], Failure]":
+        # Partial adoption of the Result contract (the ABC declares Success|Failure).
+        # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
/
+        # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused
with a
+        # genuine 0-rally video (which still returns a bare ``[]``). Every caller
+        # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
Failure)``;
+        # the bare-list success/empty paths are the documented incremental gap
(CODE_MAP gotchas).
         label = self.display_label
         # --- Sport profile + AI provider wiring (identical across segmenters) ---
         sport_name = self.config.sport
@@ -267,7 +273,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         ok, msg = runner.healthcheck()
         if not ok:
             logger.error("%s detector environment not ready: %s", label, msg)
-            return []
+            # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
+            # failed/retryable instead of writing an empty-but-"successful" result.
+            return Failure(
+                message=f"{label} detector environment not ready: {msg}",
+                is_recoverable=True,
+            )
         logger.info("%s detector env OK: %s", label, msg)

         # --- Phase 2: trajectory -> candidate rally windows ---
@@ -278,7 +289,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
         if not csv_path:
             logger.error("%s produced no trajectory; aborting.", label)
-            return []
+            # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
+            # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
+            # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
+            # rather than a silent 0-segment "success" (the wf294 incident).
+            return Failure(
+                message=(
```

```
67      +                   f"{label} detector produced no trajectory (timed out or aborted) ?
"
68      +                   "detector failure, not a 0-rally video."
69      +               ),
70      +               is_recoverable=True,
71      +           )
72
73              points = runner.parse_trajectory_csv(csv_path)
74              logger.info(
75      diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
76      index 461d7ca..88c50f6 100644
77      --- a/backend/worker/__main__.py
78      +++ b/backend/worker/__main__.py
79      @@ -114,6 +114,42 @@ def _write_report_json(
80              )
81
82
83      +def _serialize_and_push_outputs(
84      +    scratch: str,
85      +    out_uri: str,
86      +    video_id: str,
87      +    segments: List[dict],
88      +    status: str,
89      +    storage_cfg: dict,
90      +    video_uri: str = "",
91      +) -> List[str]:
92      +    """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
93      +    ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
94      +
95      +    Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
96      +    ``status="failed"`` report.json in the bucket ? the control plane's source of truth
97      +    (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
98      +    report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
99      +    EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
100     +    out_local = os.path.join(scratch, "outputs", video_id)
101     +    os.makedirs(out_local, exist_ok=True)
102     +    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
103     +    _write_report_json(
104     +        video_id,
105     +        segments,
106     +        status,
107     +        os.path.join(out_local, "report.json"),
108     +        video_uri=video_uri,
109     +    )
110     +    out_prefix = out_uri.rstrip("/")
111     +    pushed: List[str] = []
112     +    for fn in sorted(os.listdir(out_local)):
113     +        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
114     +        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
115     +        pushed.append(uri)
116     +    return pushed
117     +
118     +
119      def _run_startup_checks(config: Any) -> bool:
120          """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
121          best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
122     @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
123          )
124
```

```
125          def _fail(rc: int) -> int:
126      -       # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
127      -       # dispatcher's bucket-poll terminates FAST + accurately instead of hanging
until its poll
128      -       # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
129      +       # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
130      +       # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
131      +       # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
132      +       # truth (submit-cache probe, restart-recovery) and the alpha-user result
surface see an
133      +       # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
134      +       # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
135      +       # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
136      +       try:
137      +           _serialize_and_push_outputs(
138      +               scratch,
139      +               args.out_uri,
140      +               video_id,
141      +               [],
142      +               "failed",
143      +               storage_cfg,
144      +               str(getattr(args, "video_uri", "") or ""),
145      +           )
146      +       except Exception as e:  # never mask the real failure on an artifact-push
hiccup
147      +           logger.warning("[worker] could not push failure artifacts: %s", e)
148      +       # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
149              if getattr(args, "done_uri", None):
150                  _write_done_marker(
151                      args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
152      @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
153              )
154
155          # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
156      -   out_local = os.path.join(scratch, "outputs", video_id)
157      -   os.makedirs(out_local, exist_ok=True)
158      -   _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
159      -   _write_report_json(
160      +   pushed = _serialize_and_push_outputs(
161      +       scratch,
162      +       args.out_uri,
163              video_id,
164              segments,
165              status,
166      -       os.path.join(out_local, "report.json"),
167      -       video_uri=str(getattr(args, "video_uri", "") or ""),
168      +       storage_cfg,
169      +       str(getattr(args, "video_uri", "") or ""),
170          )
171
172      -   out_prefix = args.out_uri.rstrip("/")
173      -   pushed = []
174      -   for fn in sorted(os.listdir(out_local)):
175      -       uri = f"{out_prefix}/outputs/{video_id}/{fn}"
176      -       storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
177      -       pushed.append(uri)
178      -
```

```
179          print(
180              f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
181          )
182      diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
183      index 64909e7..c5ae442 100644
184      --- a/tests/test_tracknet_hybrid.py
185      +++ b/tests/test_tracknet_hybrid.py
186      @@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB
population/linking
187       from unittest.mock import MagicMock, patch
188
189       from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
190      +from backend.results import Failure
191
192
193       def _window(start=1.0, end=4.0):
194      @@ -124,7 +125,10 @@ def test_pipeline_healthcheck_failure_aborts(
195
196           seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
197           res = seg.process_video("v1", "clip.mp4")
198      -    assert res == []
199      +    # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
200      +    # instead of writing an empty "success".
201      +    assert isinstance(res, Failure)
202      +    assert "environment not ready" in res.message
203           # Should bail before ever running inference.
204           mock_runner_cls.return_value.run_predict.assert_not_called()
205
206      @@ -144,7 +148,36 @@ def test_pipeline_no_trajectory_aborts(
207           mock_provider_registry.get.return_value = MagicMock()
208
209           seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
210      -    assert seg.process_video("v1", "clip.mp4") == []
211      +    res = seg.process_video("v1", "clip.mp4")
212      +    # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
213      +    # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
214      +    assert isinstance(res, Failure)
215      +    assert "no trajectory" in res.message and res.is_recoverable
216      +
217      +
218      +@_patched
219      +def test_pipeline_zero_windows_is_legit_empty_not_failure(
220      +    mock_runner_cls,
221      +    mock_probe,
222      +    mock_dur,
223      +    mock_extract,
224      +    mock_provider_registry,
225      +    mock_env,
226      +):
227      +    """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
228      +    parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
229      +    (0-rally "success"), NOT a Failure."""
230      +    mock_env.return_value = "dummy_key"
231      +    mock_dur.return_value = 30.0
232      +    mock_extract.return_value = True
233      +    mock_runner_cls.return_value = _make_runner(windows=[])  # trajectory ok, zero
windows
234      +    mock_provider_registry.get.return_value = MagicMock()
235      +
```

```
    +    seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
    +    res = seg.process_video("v1", "clip.mp4")
    +    assert res == [] and not isinstance(res, Failure)
    +    # The detector DID run (a genuine empty, not an abort).
    +    mock_runner_cls.return_value.run_predict.assert_called_once()


     @_patched
    diff --git a/tests/test_worker.py b/tests/test_worker.py
    index 96edbee..63cc18e 100644
    --- a/tests/test_worker.py
    +++ b/tests/test_worker.py
    @@ -10,6 +10,7 @@ import json
     import pytest

     from backend.config import load_config
    +from backend.results import Failure
     from backend.worker import __main__ as wmain

     # ------------------------------------------------------------------ pure helpers
    @@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):


     class _EmptySeg:
    -    """A segmenter that finds nothing ? the cold-start failure mode."""
    +    """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""

         def __init__(self, db, config):
             pass
    @@ -200,6 +201,20 @@ class _EmptySeg:
             return []


    +class _FailSeg:
    +    """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
    +    trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
    +
    +    def __init__(self, db, config):
    +        pass
    +
    +    def process_video(self, video_id, path, max_frames=None):
    +        return Failure(
    +            message="WASB detector produced no trajectory (timed out or aborted)",
    +            is_recoverable=True,
    +        )
    +
    +
     def test_setup_logging_levels():
         import logging

    @@ -246,12 +261,56 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
         assert "0 segments" in msgs and "vidZ" in msgs
         assert "detector_impl=native" in msgs  # surfaces the resolved detector
         assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
    -    assert (
    -        json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
    -            "segment_count"
    +    # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
    +    # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
    +    report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
    +    assert report["segment_count"] == 0 and report["status"] == "success"
```

```
297     +
298     +
299     +def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
300     +    tmp_path, monkeypatch
301     +):
302     +    """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
303     +    as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
304     +    status="failed" so the control plane / alpha-user surface can show an error /
retry."""
305     +    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
306     +    monkeypatch.setattr(
307     +        wmain.validator_registry,
308     +        "run_all_with_context",
309     +        lambda path, cfg: ([_OKResult()], {}),
310     +    )
311     +    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
312     +    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
313     +    video = tmp_path / "in.mp4"
314     +    video.write_bytes(b"FAKE")
315     +    out = tmp_path / "bucket"
316     +    done = tmp_path / "bucket" / "_dispatch" / "tok.done"
317     +
318     +    rc = wmain.main(
319     +        [
320     +            "infer",
321     +            "--video-uri",
322     +            str(video),
323     +            "--out-uri",
324     +            str(out),
325     +            "--scratch",
326     +            str(tmp_path / "s"),
327     +            "--segmenter",
328     +            "wasb_hybrid",
329     +            "--done-uri",
330     +            str(done),
331              ]
332     -        == 0
333          )
334     +    # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
335     +    assert rc == 3
336     +    # report.json is written as an explicit FAILURE, not a false empty "success".
337     +    report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
338     +    assert report["status"] == "failed" and report["segment_count"] == 0
339     +    assert report["segments"] == []
340     +    # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
341     +    marker = json.loads(done.read_text())
342     +    assert marker["returncode"] == 3 and marker["status"] == "failed"
343     +    assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
344
345
346      def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
347
```

5. assistant (2026-07-06T17:29:12.561Z)

Now let me verify the callers actually branch on `isinstance(result, Failure)` and confirm the config-abort paths at 194/219/226 still return bare `[]`. The diff confirms lines 194, 219, 226, 231, 244 remain `return []`. Let me check the worker's cmd_infer handling and the sibling gemini segmenter.

6. user (2026-07-06T17:29:14.639Z)

```
85              """Resolve the sport profile, provider, and API key (the process_video
preamble).
86
87              Returns (sport_profile, provider) on success, or a ``Failure`` (with a user-
facing
88              message) when the run cannot proceed ? an unknown sport, a MISSING API KEY, or
an
89              unknown provider. The caller propagates the Failure so the job is marked
**failed**
90              with a reason, instead of a silent 0-rally "success" that looks like an empty
match
91              (Finding B ? the loud-failures bar). A genuine "ran, found nothing" is
unaffected.
92              """
93              # Get sport profile config (config is always typed AppConfig ?
_normalize_config()
94              # in base.py converts plain dicts passed by tests at init time)
95              sport_name = self.config.sport
96              try:
97                  sport_profile = sport_registry.get(sport_name)
98              except KeyError as e:
99                  logger.error(str(e))
100                 return Failure(str(e))
101
102             # Get AI provider and model config
103             provider_name = self.config.ai_provider
104
105             # Get appropriate API key based on the provider
106             api_key_env_var = f"{provider_name.upper()}_API_KEY"
107             api_key = os.environ.get(api_key_env_var)
108             if not api_key:
109                 from backend.pipeline.segmenters._keyhint import api_key_hint
110
111                 msg = (
112                     f"{api_key_env_var} is not set ? the {provider_name} segmenter cannot
run. "
113                     f"Set your API key. " + api_key_hint(api_key_env_var)
114                 )
115                 logger.error(msg)
116                 return Failure(msg)
117
118             try:
119                 provider = provider_registry.get(
120                     provider_name, api_key=api_key, model_name=model_name
121                 )
122             except KeyError as e:
123                 logger.error(str(e))
124                 return Failure(str(e))
125
126             return sport_profile, provider
127
128         def _maybe_trim_video(
129             self, video_id: str, video_path: str
130         ) -> Tuple[str, Callable[[], None]]:
131             """Optionally trim the video to indexing.debug_duration seconds (debug feature).
132
133             Returns (video_to_upload, cleanup_temp_files) where cleanup_temp_files removes
any
134             temp trimmed file created here.
```

7. user (2026-07-06T17:29:15.266Z)

```
300             return rc
301
302         segments: List[dict] = []
```

```
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print(
                    "[worker] validation FAILED: "
                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
                )
                return _fail(2)
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(
                    f"[worker] unknown segmenter: {seg_name!r} "
                    f"(have {list(segmenter_registry.list_available())})"
                )
                return _fail(2)
            segmenter_actual = seg_name
            result = segmenter_cls(db, config).process_video(
                video_id, local_video, max_frames=args.max_frames
            )
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return _fail(3)
            segments = result.value if hasattr(result, "value") else result
            status = "success"
            # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
            # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
            # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
            if not segments:
                detector_impl = (
                    os.environ.get("RALLY_DETECTOR_IMPL")
                    or getattr(config.indexing, "detector_impl", None)
                    or "auto"
                )
                logger.warning(
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
                    "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
                    "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
                    "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
                    "detections, or the input resolution differs from the tuned proxy
resolution. "
                    "Re-run with -v for the native command + frame counts.",
                    video_id,
                    segmenter_actual,
                    detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db,
                video_id=video_id,
```

```
358                 video_path=local_video,
359                 config=config,
360                 val_results=val_results,
361                 val_context=val_context,
362                 segmenter_requested=args.segmenter,
363                 segmenter_actual=segmenter_actual,
364                 was_reingest=False,
365                 segmentation_ran=segmentation_ran,
366             )
367 
368         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
369         pushed = _serialize_and_push_outputs(
370             scratch,
371             args.out_uri,
372             video_id,
373             segments,
374             status,
375             storage_cfg,
376             str(getattr(args, "video_uri", "") or ""),
377         )
378 
379         print(
```

8. assistant (2026-07-06T17:29:31.872Z)

The finding is fully confirmed by the code. Let me trace it precisely:

The claim: trajectory_hybrid.py's missing-API-key path (line 219), unknown-sport (194), unknown-provider (226) still `return []`, and the worker treats a bare `[]` as `status="success"`.

Verified against real code:

1. **Line 219 returns bare `[]`** ? confirmed in the Read at lines 211-219: `if not api_key: ... return []`. Same for line 194 (unknown sport, KeyError) and line 226 (unknown provider, KeyError). The diff shows the wf294 fix converted ONLY the healthcheck path (was line ~276?now line 278) and the no-trajectory path (line ~291?296) to `Failure`. Lines 194, 219, 226, 231, 244 remained bare `[]`.

2. **The worker branches on `isinstance(result, Failure)`** ? confirmed at `backend/worker/__main__.py:326`: `if isinstance(result, Failure): ... return _fail(3)`. A bare `[]` is NOT a `Failure`, so it falls through to line 330 `status = "success"`.

3. **A bare `[]` becomes `status="success"` with `segment_count=0`** ? confirmed. At line 334 `if not segments:` it logs the loud warning, but `status` stays `"success"`. Then line 369 `_serialize_and_push_outputs(... segments, status ...)` writes `report.json` with `status="success"`.

4. **The worker's OWN diagnostic names this exact case** ? confirmed at lines 343-345: "a missing ${AI_PROVIDER}_API_KEY with skip_ai_handoff=False (the segmenter bails BEFORE detection...)". So the code itself acknowledges the missing-key case bails before detection and surfaces as a 0-segment success.

5. **The sibling gemini segmenter treats these three conditions as `Failure`** ? confirmed at `gemini.py:100` (unknown sport), `:116` (missing key), `:124` (unknown provider), all `return Failure(...)`, with a docstring citing "not a silent 0-rally 'success' that looks like an empty match".

Reproduction (as written): A `wasb_hybrid` job with `skip_ai_handoff=False` and `GEMINI_API_KEY` unset ? line 211 `if not api_key` is True ? line 219 `return []` ? worker line 326 `isinstance([], Failure)` is False ? line 330 `status="success"` ? `report.json` written with `status="success"`, `segment_count=0`. This is exactly the wf294 silent-failure shape the fix set out to close, and it is inconsistent with both the gemini sibling and the two paths the same diff DID convert.

The severity assessment (high) is reasonable: the missing-key case is a common real misconfig (env not set before a cloud run), it is silent (indistinguishable from a legit empty match by the control plane), and the fix explicitly targeted this class but left the most acute pre-detection path unconverted. This is a genuine completeness gap in the fix, not out-of-scope speculation ? the diff's own converted paths establish the intended convention.

9. user (2026-07-06T17:29:38.524Z)

Structured output provided successfully